about the signature is in our IDE but is not used to the fullest extent for some reason. The same is happening with the additional argument. The reason it does not highlight anything is due to some minor bug.

Even a simple decorator can confuse IDE hinting, but in real life we may want to implement some more difficult decorators that we would not be able to annotate properly to help IDE to cope with this.

Let us go back to our sleeping module for a second, and say that we got some feedback from our users to ensure our function is good. The users do not want to import time units every time we use the module. They want to be able to use both TimeUnit enum and the string literal. So, we write another decorator called *autoenum*, which will take the function, analyse the signature, and pick the arguments that are annotated as enum types. For those particular arguments, we will try to cast all the arguments that pass to the enum value. The code will look something like what is shown in Figure 6.

The process is quite complicated, but let us go through it. First, we take the original signature of our function. Next, we bind our arbitrary arguments *args* and *kwargs* to the original signature in order to understand which values are passed to which argument name. Then we traverse the pairs of argument names and values passed to this argument. In case the argument was annotated as enum in the original signature, and if for some reason the value passed to this argument is not an enum, then we can assign a new argument, which is an argument annotation of original value. It takes an enum class and creates an enum instance from a string with a row name.

There is a big chunk of code that preserves the signatures. If we want our new function to accept both enums and string literals, then we must update the annotations so that every enum annotation is replaced with a union of enums and some string literals. We can recreate a signature object and assign it to a signature attribute.

In Figure 7, we can see that our IDE highlights the MINUTE string as an invalid argument, but if we run the code we can see that both versions of our code, i.e., the one with the time unit and the one with the string, are actually working. And when we try to get the

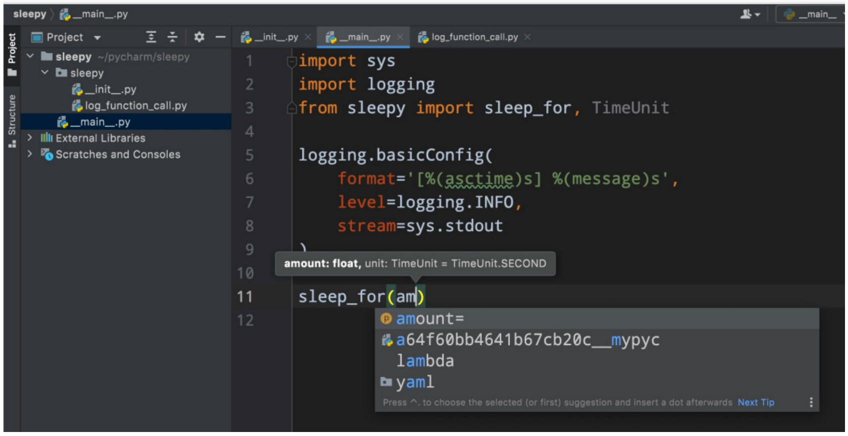

Figure 5: Pop-up window with list

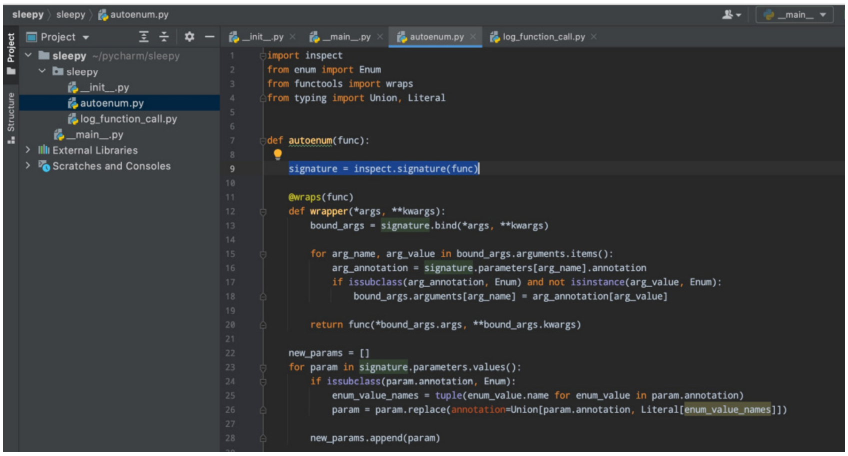

Figure 6: The code

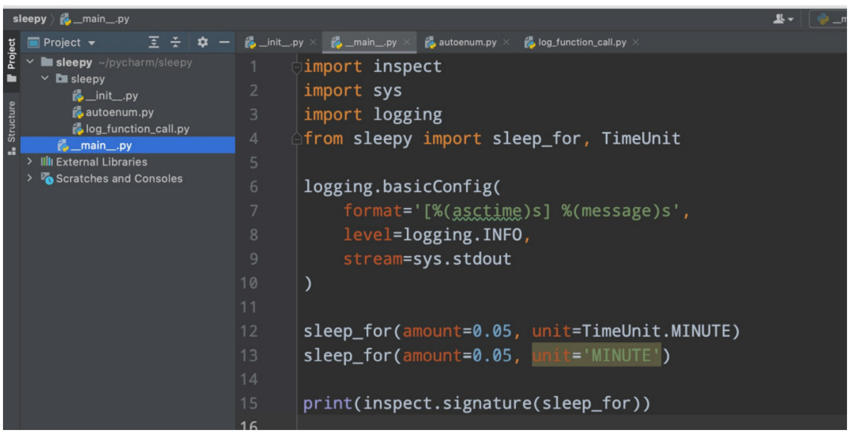

Figure 7: Updated IDE